

The Oracle: Un enfoque híbrido para la comparación de scripts de Python mediante métricas textuales y análisis semántico con LLM

*Matias Negrelli, Nicolás Hernández, Emanuel Ravera
Laboratorio de Investigación de Software
Departamento de Ingeniería en Sistemas de Información
Facultad Regional Córdoba - Universidad Tecnológica Nacional.*

Resumen

Este trabajo presenta The Oracle, una aplicación web disponible en <https://theoracle-three.vercel.app>. La herramienta compara pares de scripts de Python para estimar su similitud mediante tres algoritmos clásicos de análisis de texto: Term Frequency [1] con similitud del coseno, Índice de Jaccard [2] y distancia de Levenshtein [3]. Además, complementa el análisis con una inspección semántica provista por la API¹ (Application Programming Interface) de Gemini.

The Oracle entrega tres resultados principales. Primero, un puntaje de similitud según la métrica seleccionada. Segundo, un reporte cualitativo que incluye las técnicas de modificación detectadas y dos descripciones generadas por Gemini: una sobre la implementación y otra sobre la funcionalidad. Tercero, una función opcional de reescritura de código que crea variantes del script original bajo instrucciones del usuario.

Los resultados experimentales muestran que este enfoque híbrido (algoritmos + IA) mejora la interpretabilidad y ofrece diagnósticos más robustos frente a modificaciones superficiales. La arquitectura de la aplicación se apoya en el protocolo AG-UI² (Agente – Interfaz de Usuario), un diseño que separa la lógica algorítmica de la capa visual para asegurar modularidad, escalabilidad y mantenibilidad.

1. Introducción

La comparación automática de código fuente es un campo con gran relevancia tanto en ámbitos académicos como industriales. En la educación, los sistemas de detección de plagio permiten preservar la integridad académica y dar retroalimentación a estudiantes sobre la originalidad de sus soluciones. En la industria, la detección de clones y la identificación de variantes de software contribuyen al mantenimiento de grandes bases de código, reducción de redundancia y prevención de errores³.

¹ <https://ai.google.dev/gemini-api/docs?hl=es-419>

² <https://webflow.copilotkit.ai/blog/intro-to-ai-agents>

³ https://www.ugr.es/~plagio_hum/Documentacion/06Publicaciones/ART003.pdf

El problema central radica en que los scripts de programación pueden ser modificados de múltiples formas superficiales, cambios en comentarios, formato, renombrado de variables que no alteran la funcionalidad, pero confunden a los algoritmos tradicionales de comparación. Al mismo tiempo, existen transformaciones más profundas, como la reordenación de bloques o la sustitución de estructuras de control, que pueden alterar significativamente la estructura sintáctica sin necesariamente modificar la semántica. Por ello, un sistema robusto debe ser capaz de distinguir entre cambios superficiales y cambios que afectan la implementación o la funcionalidad.

2. Problema

Dado un par de scripts de Python, estimar su similitud cuantitativa y explicar de manera cualitativa qué tipo de cambios se aplicaron. La solución debe superar las limitaciones de las métricas puramente textuales, que en ciertos contextos sobreestiman o subestiman la similitud.

3. Hipótesis

La combinación de métricas textuales (TF+coseno, Jaccard, Levenshtein) con un análisis semántico asistido por un LLM mejora la estabilidad de los puntajes ante transformaciones triviales, la capacidad para discriminar entre modificaciones cosméticas y funcionales, y la interpretabilidad de los resultados para el usuario final.

Se postula además que la integración de una funcionalidad de reescritura controlada de código permitirá generar datasets sintéticos y casos de prueba que validen mejor el comportamiento de las métricas.

Para abordar esta problemática se desarrolló The Oracle, una aplicación web que implementa un pipeline híbrido. La solución incorpora tres algoritmos textuales clásicos, complementados con los servicios de análisis y reescritura de la API de Gemini, enmarcados bajo el denominado Protocolo AG-UI. Esta combinación busca no solo calcular porcentajes de similitud, sino también proveer explicaciones interpretables y generar variantes de código para experimentación controlada.

4. Protocolo AG-UI

El Protocolo AG-UI es un enfoque arquitectónico diseñado para aplicaciones donde la lógica algorítmica debe estar desacoplada de la presentación visual. Su premisa principal es que los algoritmos centrales (en este caso, los de similitud y análisis de texto) se implementen y validen de forma independiente, mientras que la interfaz gráfica se limita a invocar, mostrar y facilitar la interacción con dichos algoritmos.

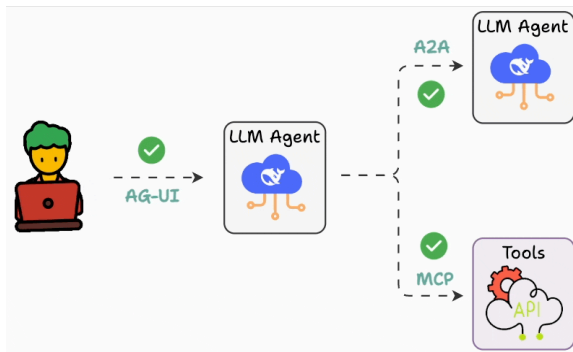


Figura 2. Protocolo AG-UI

Características principales del protocolo:

- **Separación de responsabilidades:** los algoritmos se agrupan en un módulo dedicado (AG), mientras que la UI se centra exclusivamente en la experiencia del usuario.
- **Independencia tecnológica:** el núcleo algorítmico puede desarrollarse en un lenguaje o framework distinto al de la UI. En The Oracle, los algoritmos están en JavaScript y la interfaz en React + Tailwind + TypeScript [5].
- **Escalabilidad:** permite incorporar nuevos algoritmos de comparación sin afectar la interfaz. El protocolo estandariza la comunicación entre AG y UI.
- **Pruebas y validación:** el núcleo AG puede ser testeado de forma unitaria con datasets sintéticos, sin necesidad de levantar toda la interfaz.
- **Reutilización:** el módulo AG puede integrarse en otras aplicaciones o entornos distintos con mínimos cambios.

5. Beneficios para The Oracle

- **Mantenibilidad:** los algoritmos (TF+coseno, Jaccard, Levenshtein) están encapsulados en un módulo reutilizable.
- **Extensibilidad:** se pueden añadir algoritmos estructurales (AST, CFG) sin modificar la interfaz.

- **Consistencia visual:** la UI permanece estable, mostrando resultados y reportes de Gemini de forma uniforme.
- **Optimización del flujo de trabajo:** al seguir el patrón, la API de Gemini se conecta directamente al módulo AG, que procesa y devuelve los resultados ya preparados para la U

6. Comparación con otros patrones

A diferencia de MVC (Modelo-Vista-Controlador) o MVVM (Modelo-Vista-Modelo de Vista)⁴, que centran la separación en datos, vistas y controladores, Protocolo AG-UI se enfoca en el aislamiento del motor algorítmico frente a la interfaz. Esto lo hace más adecuado para aplicaciones de análisis comparativo como The Oracle, donde la lógica de procesamiento de datos es intensiva y evoluciona más rápido que la capa visual.

7. Metodología (Algoritmos de similitud)

Term Frequency + Similitud del Coseno: El algoritmo implementado en The Oracle tokeniza ambos scripts y genera vectores de frecuencia de términos (TF). La tokenización se realiza aplicando una expresión regular que extrae secuencias alfanuméricas y guiones bajos (`(\b\w+\b)`), transformando previamente todo el texto a minúsculas. Esto asegura que identificadores como “Variable”, “variable” o “VARIABLE” se consideren equivalentes, y elimina el ruido de símbolos de puntuación o espacios. El resultado es una lista normalizada de tokens que alimenta el cálculo de frecuencias.

Cada vector se normaliza dividiendo las ocurrencias de cada token por la longitud total del texto. La similitud se obtiene calculando el coseno entre ambos vectores, definido como el cociente entre el producto punto y el producto de las magnitudes.

Inicialmente se consideró el uso de TF-IDF (Term Frequency - Inverse Document Frequency), pero se determinó que era erróneo aplicarlo en este contexto, ya que TF-IDF se utiliza para ponderar términos en un corpus amplio, mientras que aquí se trata de una comparación directa entre dos scripts. El factor IDF pierde sentido y distorsiona los resultados. Por ello, The Oracle emplea únicamente TF normalizado + coseno.⁵

Índice de Jaccard: Implementado mediante la construcción de conjuntos de tokens para cada script

⁴<https://dev.to/chiragagg5k/architecture-patterns-for-beginners-mvc-mvp-and-mvvm-2pe7>

⁵<https://medium.com/@anurag-jain/tf-idf-vectorization-with-cosine-similarity-eca3386d4423>

La similitud se calcula como el tamaño de la intersección dividido por el tamaño de la unión. La simplicidad del algoritmo lo hace eficiente, pero al ignorar la frecuencia y el orden de aparición, se vuelve vulnerable a cambios superficiales (comentarios, formato o redundancias).⁶

Levenshtein: The Oracle implementa la distancia de Levenshtein normalizada, que mide el número mínimo de operaciones de edición (inserción, eliminación o sustitución de caracteres) necesarias para transformar un script en otro. Posteriormente, la similitud se calcula como 1 menos la distancia relativa. Aunque captura modificaciones locales con gran sensibilidad, penaliza excesivamente los reordenamientos de bloques grandes de código.⁷ [4]

8. Análisis semántico con LLM

La API de Gemini recibe tanto el código original como el modificado y mediante un prompt previamente diseñado produce:

- Técnicas de modificación detectadas.
- Un análisis breve sobre la equivalencia funcional.
- Una evaluación de diferencias de implementación.

Este módulo complementa los resultados cuantitativos, ofreciendo una capa explicativa que cubre los fallos de los algoritmos clásicos.

La versión utilizada de la API de Gemini es Gemini 2.5 Flash, un modelo de lenguaje de última generación optimizado para tareas de análisis de código y generación estructurada de respuestas.

9. Resultados

La siguiente tabla resume el comportamiento de cada métrica frente a diferentes técnicas de modificación:

Técnica de modificación	TF-SC	Jaccard	Levenshtein	IA
Modificación de comentarios	90.18%	51.82%	78.91%	✓
Cambio de formato	100%	100%	96.98%	✓
Renombrado	90.41%	81.37%	90.58%	✓
Reordenación de código	100%	100%	53.85%	✓
Cambios de tipos de datos	99.61%	97.87%	98.47%	✓
instrucciones redundantes	91.05%	68%	76.60%	✓
Estructuras de control equivalentes	96.13%	91.01%	85.08%	✓
Modificación de funcionalidad	93.74%	82.14%	72.24%	✓

Tabla 1. Resultados de The Oracle

Observaciones:

- TF+coseno se mostró estable en todos los casos.
- Jaccard fue sensible a modificaciones superficiales como comentarios o redundancias.
- Levenshtein penaliza fuertemente la reordenación de código.
- La IA detectó correctamente todas las técnicas en el 100% de los casos ✓.
- En situaciones donde las métricas textuales arrojaron resultados engañosos o inconsistentes (levenshtein en el caso de “reordenación de código” y Jaccard en “Modificación de comentarios”), la IA corrigió el fallo al identificar la modificación real y describir su impacto.

⁶<https://www.grapheverywhere.com/algoritmo-de-similaridad-de-jaccard/>

⁷<https://medium.com/@tejaswiadav221/the-levenshtein-distance-algorithm-a-string-metric-for-measuring-the-difference-between-two-269afbbddd34>

10. Discusión

Los resultados confirman la hipótesis: un análisis híbrido mejora la interpretación. TF+coseno se comportó de manera robusta, mientras que Jaccard y Levenshtein tienen limitaciones según el tipo de transformación. La IA añadió valor explicando el tipo exacto de modificación, algo que las métricas numéricas por sí solas no pueden lograr.

Fallos de los algoritmos y cobertura de la IA:

TF + Coseno: falla en casos de reordenamiento, donde la proporción de tokens permanece igual y la métrica indica similitud plena sin reflejar cambios estructurales.

Jaccard: falla ante modificaciones superficiales, como añadir comentarios, penalizando de manera desproporcionada.

Levenshtein: falla con reordenamientos grandes, interpretándose como diferencias absolutas.

En contraste, la IA siempre identificó la técnica aplicada, incluso cuando los algoritmos entregaban valores engañosos. Esto la convierte en un complemento indispensable para el sistema, aportando precisión interpretativa y explicaciones cualitativas.

11. Limitaciones

Aunque The Oracle combina métricas textuales y un modelo LLM, también existen limitaciones que deben reconocerse. En primer lugar, el uso de la API de Gemini implica una dependencia de un proveedor externo, lo que condiciona la continuidad del servicio y los costos de uso en escenarios de gran escala. Además, como todo modelo de lenguaje, Gemini puede heredar sesgos de sus datos de entrenamiento y producir análisis que reflejen tales sesgos. Finalmente, la ejecución de múltiples validaciones o reescrituras puede resultar costosa en términos de recursos computacionales y financieros, lo que limita su aplicación masiva en contextos educativos con gran número de estudiantes o en auditorías extensivas.

Estas restricciones señalan la necesidad de explorar alternativas complementarias, como modelos locales o híbridos que reduzcan la dependencia y los costos.

12. Conclusiones

El aporte principal de este trabajo es demostrar que la integración de LLMs dentro de un pipeline de comparación de código permite superar limitaciones históricas de las métricas textuales. Mientras que algoritmos como TF + coseno, Jaccard o Levenshtein proporcionan medidas cuantitativas valiosas, suelen fallar frente a modificaciones superficiales o reordenamientos estructurales. La incorporación de un modelo de lenguaje

avanzado como Gemini 2.5 Flash añade la capa semántica necesaria para interpretar los cambios, corregir los sesgos de las métricas y ofrecer explicaciones claras sobre la equivalencia funcional y la implementación.

Este enfoque híbrido sienta las bases para un nuevo paradigma en el análisis automático de código: uno que combina precisión numérica con comprensión semántica. De esta manera, se logran no solo puntajes de similitud más robustos, sino también diagnósticos interpretables que enriquecen la toma de decisiones en educación, auditoría y desarrollo de software.

De cara al futuro, la arquitectura basada en el patrón AG-UI abre múltiples posibilidades: soportar nuevos lenguajes de programación y generar datasets sintéticos para entrenar y evaluar algoritmos. Con estas líneas de trabajo, The Oracle se proyecta como una herramienta capaz de evolucionar hacia sistemas de comparación más confiables, transparentes y adaptados a las necesidades reales de la ingeniería de software.

13. Futuras expansiones de The Oracle

The Oracle tiene un camino prometedor hacia su expansión. Los planes futuros se centran en mejorar su capacidad de análisis y aplicabilidad.

Análisis a nivel de proyecto: Extender el análisis de scripts individuales a proyectos completos, lo que implicaría comparar múltiples archivos, sus dependencias y la interacción entre ellos. Esto requerirá el desarrollo de algoritmos más complejos que analicen la estructura del software, no solo el contenido de los archivos.

Integración de nuevos algoritmos: Se plantea la integración de algoritmos de comparación estructural, como aquellos basados en Árboles de Sintaxis Abstracta (AST) y Grafos de Flujo de Control (CFG). Un AST es una representación jerárquica de la estructura del código que abstrae los detalles sintácticos, mientras que un CFG representa todos los caminos que la ejecución del programa podría seguir. Estos algoritmos serían mucho más robustos frente a modificaciones superficiales, ya que se centran en la estructura lógica del código y no en su representación textual.

Soporte multilinguaje: Actualmente, The Oracle está enfocado en Python. Expandir el soporte a otros lenguajes de programación sería un siguiente paso natural para aumentar la utilidad de la herramienta en diferentes contextos académicos e industriales.

Uso de modelos locales y alternativos: Se proyecta evaluar el empleo de modelos de lenguaje ejecutados localmente o en entornos híbridos. Esto permitiría reducir la dependencia de proveedores externos como Gemini, minimizar costos y ofrecer mayor control sobre privacidad y personalización. Asimismo, se contempla la incorporación de otros modelos en la nube, como ChatGPT o Claude, capaces de realizar las mismas funciones que Gemini (detección de técnicas, análisis de

equivalencia y reescritura de código). De este modo, Oracle podría diversificar su infraestructura de análisis y evitar depender de un único proveedor.

14. Referencias

- [1] Moyotl-Hernández, E., & Jiménez-Salazar, H. (2004). An analysis on Frequency of terms for Text Categorization. *Procesamiento del lenguaje natural*, 33.
- [2] Abdullah, S. M., Ali, S. M., & Makttof, M. A. (2019). Modifying Jaccard coefficient for texts similarity. *Opción: Revista de Ciencias Humanas y Sociales*, (19), 28.
- [3] Mego Lizana, J. A., & Cespedes Bravo, S. M. Algoritmo para la corrección de textos en español basados en los algoritmos Metaphone y Distancia de Levenshtein.
- [4] Amón, I., & Jiménez, C. (2010). Funciones de similitud sobre cadenas de texto: una comparación basada en la naturaleza de los datos. In *CONF-IRM Proceedings* (Vol. 58).
- [5] MatiasSoft. (2025, 13 de agosto). The-Oracle [Repositorio de código]. GitHub.
<https://github.com/MatiasSoft/The-Oracle>

Anexo

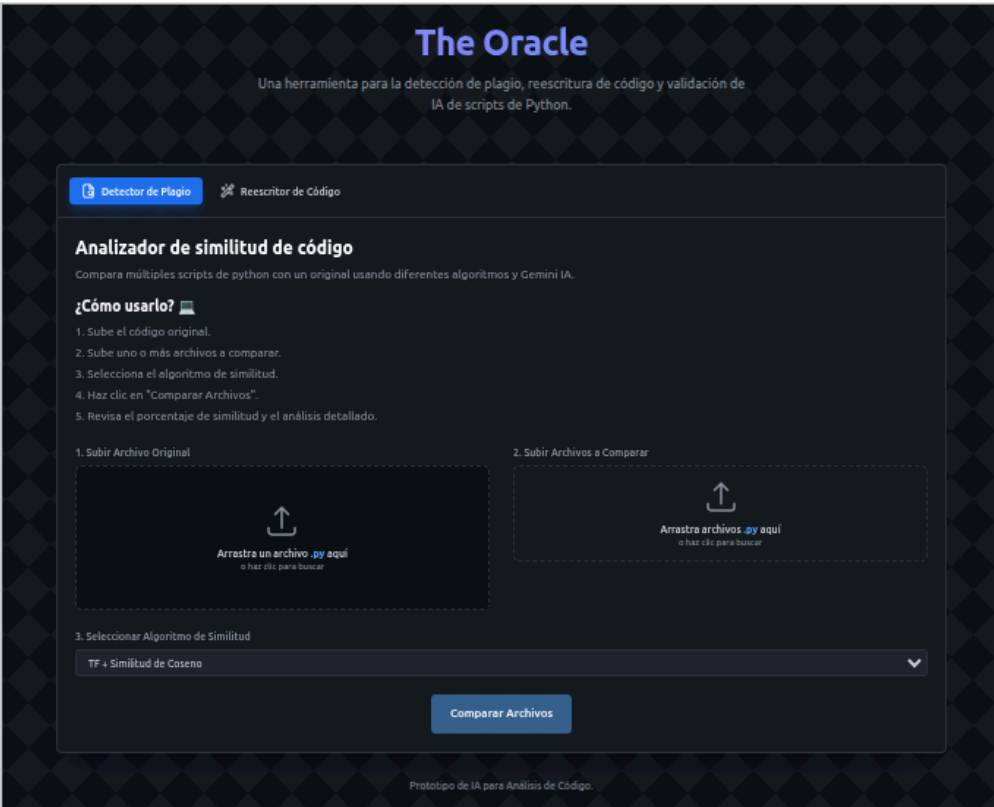


Figura 2. Interfaz The Oracle.

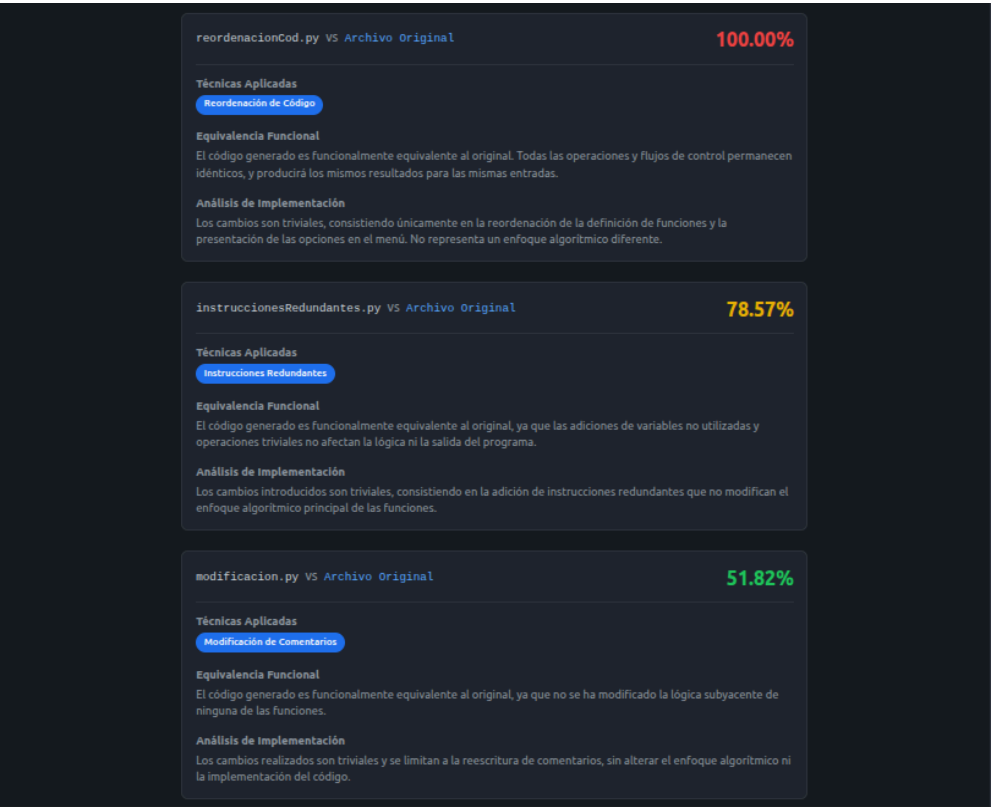


Figura 3. Resultados de The Oracle.